Alpha-Beta-Pruning

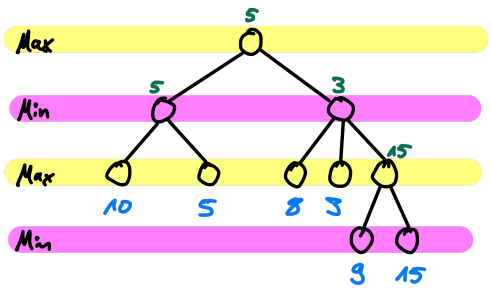
Carsten Gips (HSBI)

TL;DR
Minimax entwickelt den gesamten Spielbaum. Wenn man dabei die bisher besten Werte für MAX und MIN als α und β mitführt, beobachtet man, dass ein α-Wert nie kleiner wird und ein β-Wert nie größer wird. Dies kann man ausnutzen und das Entwickeln von Pfaden abbrechen, wenn in einem MIN-Knoten der β-Wert kleiner wird als der α-Wert des MAX-Vorgängers: (a) kann der β-Wert bei der weiteren Untersuchung der verbleibenden Nachfolger im MIN-Knoten nur noch kleiner werden, und (b) würde der MAX-Vorgänger diesen MIN-Knoten nie als Nachfolger in Betracht ziehen, da er bereits einen besseren Zug gesehen hat (da sein α größer ist als das β im Nachfolger). Deshalb kann man hier sofort die Untersuchung der verbleibenden Nachfolger im MIN-Knoten abbrechen ("Pruning"). Eine analoge Überlegung gilt für einen MAX-Nachfolger unter einem MIN-Knoten.
Dabei bleibt das Endergebnis erhalten. Man schneidet nur Suchpfade weg, die das Ergebnis von Minimax nicht verändern.
Die mögliche Effizienzsteigerung ist sehr stark abhängig von Sortierung der Nachfolger! Deshalb stellt man häufig Heuristiken zur "richtigen" Sortierung der Nachfolger auf ("Killer-Moves").
Zusätzlich kann man wie bei Minimax auch die Suchtiefe begrenzen und eine Bewertungsfunktion (statt der Nutzenfunktion) einsetzen. Auch hier kann die Bewertungsfunktion wieder gewichtete Features nutzen und/oder Positionen mit in Datenbanken gespeicherten Positionen und Bewertungen abgleichen.
Videos

- VL Alpha-Beta-Pruning

---

1 Verbesserung Minimax-Algorithmus



=> **Minimax-Baum**: Verbesserungen möglich?

2 Alpha-beta-Pruning

Minimax-Algorithmus mit zusätzlichen Informationen:

- α: bisher bester Wert für MAX (höchster Wert)
- β: bisher bester Wert für MIN (kleinster Wert)

=> Beobachtungen:

1. α für MAX-Knoten wird nie kleiner
2. β für MIN-Knoten wird nie größer

3 Pruning-Regeln

1. Schneide (unter) MIN-Knoten ab, deren $\beta \leq$ dem α des MAX-Vorgängers ist.

2. Schneide (unter) MAX-Knoten ab, deren $\alpha \geq$ dem β des MIN-Vorgängers ist.

Abbruch, wenn kein Platz mehr zwischen Alpha und Beta

4 Alpha-beta-Pruning – Der Algorithmus (Skizze)

```
def Max-Value(state, alpha, beta):
    if Terminal-Test(state): return Utility(state)

    v = -INF
    for (a, s) in Successors(state):
        v = MAX(v, Min-Value(s, alpha, beta))
        if (v >= beta): return v
        alpha = MAX(alpha, v)
    return v
```

```
def Min-Value(state, alpha, beta):
    if Terminal-Test(state): return Utility(state)

    v = +INF
    for (a, s) in Successors(state):
        v = MIN(v, Max-Value(s, alpha, beta))
        if (v <= alpha): return v
        beta = MIN(beta, v)
    return v
```

Initialer Aufruf von `Max-Value()` mit $\alpha = -\infty$ und $\beta = +\infty$

Achtung: Es kursieren Varianten von diesem Algorithmus, bei denen auf die Hilfsvariable `v` verzichtet wird und stattdessen `alpha` bzw. `beta` direkt modifiziert werden und als Rückgabewert dienen. Das *kann* zu anderen oder falschen Ergebnissen führen! Sie können das in der Aufgabe auf Blatt 03 gut beobachten.

5 Alpha-beta-Pruning – Eigenschaften

1. Pruning beeinflusst nicht das Endergebnis!

2. Sortierung der Nachfolger spielt große Rolle
3. Perfekte Sortierung: $O(b^{d/2})$ => Verdopplung der Suchtiefe möglich

Für Schach immer noch zu aufwändig …

6 Verbesserungen für Alpha-beta-Pruning

- "Killer-Move": Maximale Effizienz nur wenn **optimaler Zug immer zuerst** untersucht
  => Zu untersuchende Züge **sortieren/priorisieren**, zb. Schach:
  a) Figuren schlagen
  b) Drohen
  c) Vorwärts ziehen
  d) Rückwärts ziehen
- Verändern der Suchtiefe nach Spielsituation
- Bewertungsfunktion `Eval`:
  - Datenbanken mit Spielsituationen und Expertenbewertung:
    * Eröffnungsspiele (besonders viele Verzweigungen)
    * Endspiele
  - Lernen der optimalen Gewichte für `Eval`-Funktion
  - Berücksichtigung von Symmetrien

7 Beispiel DeepBlue (IBM, 1997)

- Alpha-beta-Pruning mit Tiefenbeschränkung: ca. 14 Halbzüge
- Dynamische Tiefenbeschränkung (stellungsabhängig, max. ca. 40 Züge)
- Heuristische Stellungsbewertung `Eval`:
  - mehr als 8.000 Features
  - ca. 4.000 Eröffnungsstellungen
  - ca. 700.000 Spielsituationen (von Experten bewertet)
  - Endspiel-Datenbank: alle Spiele mit 5 Steinen, viele mit 6 Steinen

Quelle: (Russell und Norvig 2014, p. 185)

8 Beispiel AlphaGo (Google, 2016)

- Beschränkung der Suchtiefe: Bewertung der Stellung durch *"Value Network"*

- Beschränkung der Verzweigungsbreite: Bestimmung von Zugkandidaten durch *"Policy Network"*
- Training dieser *"Deep Neural Networks"*:
  - Überwachtes Lernen: "Analyse" von Spiel-Datenbanken
  - Reinforcement-Lernen: Self-Play, Bewertung am Ende
    - Züge mit Monte-Carlo-Baumsuche ausgewählt

Quelle: (Silver u. a. 2016), siehe auch deepmind.com/research/alphago/

9 Wrap-Up

- Alpha-beta-Pruning:
  - Mitführen der bisher besten Werte für MAX und MIN: α und β
  - Abschneiden von Pfaden, die Verschlechterung bewirken würden
  - Endergebnis bleibt erhalten
  - Effizienzsteigerung abhängig von Sortierung der Nachfolger
- Viele Verbesserungen denkbar:
  - Zu untersuchende Züge "richtig" sortieren (Heuristik)
  - Suchtiefe begrenzen und Bewertungsfunktion (statt Nutzenfunktion)
  - Positionen mit Datenbank abgleichen

10 Zum Nachlesen

- Russell und Norvig (2020): Alpha-beta-Pruning: Abschnitt 6.2.3, Erweiterungen: Abschnitt 6.3
- Ertel (2017)

Lernziele

- k2: Optimierungsmöglichkeit: Sortierung der Nachfolger => Heuristik
- k2: Optimierungsmöglichkeit: Suchtiefe beschränken => Übergang zu Bewertungsfunktion
- k2: Optimierungsmöglichkeit: Bewertung über Spieldatenbanken
- k3: alpha-beta-Pruning

Quizzes

- Selbsttest Alpha-Beta-Pruning (ILIAS)

Challenges

Optimale Spiele und MiniMax

Auf einem Tisch liegen nebeneinander 5 Streichhölzer. Es gibt zwei Spieler - Weiß und Schwarz - die abwechselnd ein oder zwei Streichhölzer wegnehmen dürfen (es muss mind. ein Streichholz genommen werden). Wer das letzte Streichholz nehmen muss, hat verloren. Zu Beginn ist Weiß am Zug.

1. Spielbaum

Zeichnen Sie den **kompletten** Spielbaum auf. Geben Sie an den Kanten jeweils die Zahl der genommenen und der verbleibenden Hölzer an.
Beispiel: Wenn in einem Zug ein Holz genommen wird und 3 Hölzer verbleiben, steht an der entsprechenden Kante "1/3". Geben Sie jeweils an, welcher Spieler am Zug ist.

2. Minimax

Geben Sie die Bewertung aller Spielzustände mit Hilfe des Minimax-Algorithmus an. Bewerten Sie die Endzustände mit +1, wenn Spieler Weiß gewonnen hat, mit -1, falls Schwarz gewonnen hat.

3. Alpha-Beta-Pruning

Wenden Sie Alpha-Beta-Pruning auf den Spielbaum an. Werden damit mehr oder weniger oder gleich viele Spielzüge wie mit Minimax entwickelt? Begründen Sie Ihre Antwort.

Quellen

Ertel, W. 2017. *Introduction to Artificial Intelligence*. 2nd edition. Springer. https://doi.org/10.1007/978-3-319-58487-4.

Russell, S., und P. Norvig. 2014. *Artificial Intelligence: A Modern Approach*. 3rd revised edition. Pearson. http://aima.cs.berkeley.edu.

———. 2020. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson. http://aima.cs.berkeley.edu.

Silver, D., A. Huang, C. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, I. Schrittwieser J. Schrittwieser, V. Panneershelvam, M. Lanctot, u. a. 2016. „Mastering the Game of Go with Deep Neural Networks and Tree Search". *Nature* 529 (7587): 484. https://doi.org/10.1038/nature16961.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- (Silver u. a. 2016), siehe auch deepmind.com/research/alphago/
- (Russell und Norvig 2014, p. 185)

> **Last modified:** 80d2904 (Remove Hugo: remove cbox, 2025-07-17)